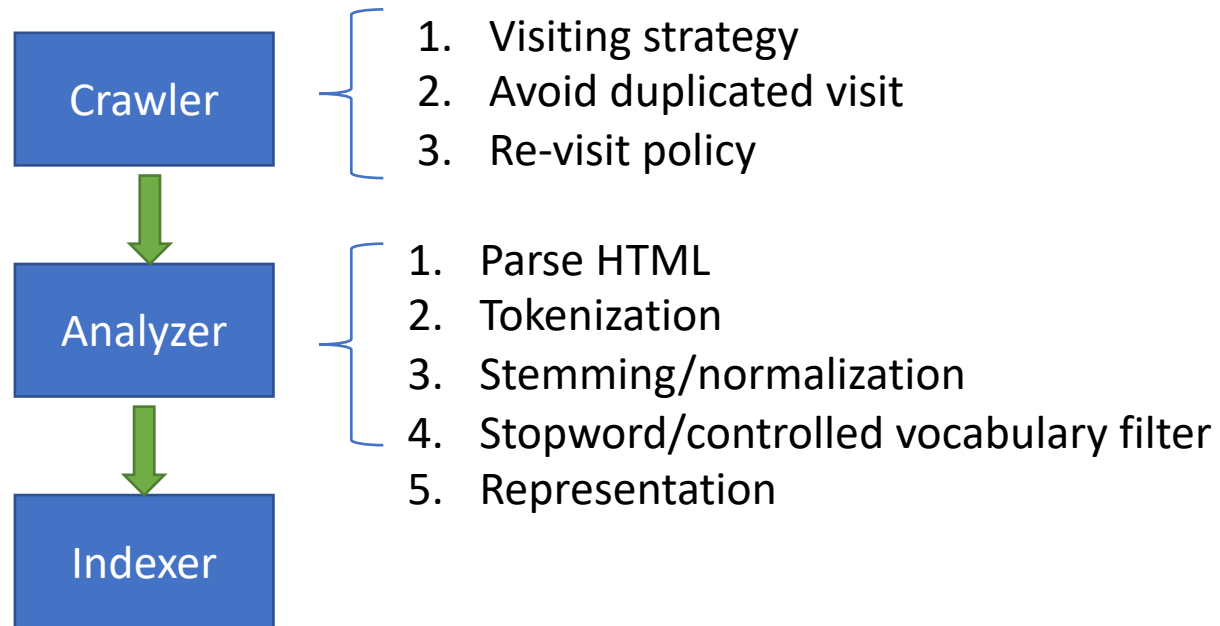


Lecture 3: Inverted Index and Index Construction

Huazheng Wang

EECS@OSU

Recap: search engine architecture



Recap: text indexing

- Bag-of-Words representation

Terms	Documents													
	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12	M13	M14
abnormalities	0	0	0	0	0	0	0	1	0	1	0	0	0	0
age	1	0	0	0	0	0	0	0	0	0	0	1	0	0
behavior	0	0	0	0	1	1	0	0	0	0	0	0	0	0
blood	0	0	0	0	0	0	0	1	0	0	1	0	0	0
close	0	0	0	0	0	0	1	0	0	0	1	0	0	0
culture	1	1	0	0	0	0	0	1	1	0	0	0	0	0
depressed	1	0	1	1	1	0	0	0	0	0	0	0	0	0
discharge	1	1	0	0	0	1	0	0	0	0	0	0	0	0
disease	0	0	0	0	0	0	0	0	1	0	1	0	0	0
fast	0	0	0	0	0	0	0	0	0	1	0	1	1	1
generation	0	0	0	0	0	0	0	0	1	0	0	0	1	0
oestrogen	0	0	1	1	0	0	0	0	0	0	0	0	0	0
patients	1	1	0	1	0	0	0	1	0	0	0	0	0	0
pressure	0	0	0	0	0	0	0	0	0	0	1	0	0	1
rats	0	0	0	0	0	0	0	0	0	0	0	0	1	1
respect	0	0	0	0	0	0	0	1	0	0	0	1	0	0
rise	0	0	0	1	0	0	0	0	0	0	0	0	0	1
study	1	0	1	0	0	0	0	0	1	0	0	0	0	0

- Doc1: Information retrieval is helpful for everyone.
- Doc2: Helpful information is retrieved for you.

	information	retrieval	retrieved	is	helpful	for	you	everyone
Doc1	1	1	0	1	1	1	0	1
Doc2	1	0	1	1	1	1	1	0

Word-document adjacency matrix

Complexity analysis

Terms	Documents													
	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12	M13	M14
abnormalities	0	0	0	0	0	0	0	1	0	1	0	0	0	0
age	1	0	0	0	0	0	0	0	0	0	0	1	0	0
behavior	0	0	0	0	1	1	0	0	0	0	0	0	0	0
blood	0	0	0	0	0	0	0	1	0	0	1	0	0	0
close	0	0	0	0	0	0	1	0	0	0	1	0	0	0
culture	1	1	0	0	0	0	0	1	1	0	0	0	0	0
depressed	1	0	1	1	1	0	0	0	0	0	0	0	0	0
discharge	1	1	0	0	0	1	0	0	0	0	0	0	0	0
disease	0	0	0	0	0	0	0	0	1	0	1	0	0	0
fast	0	0	0	0	0	0	0	0	0	1	0	1	1	1
generation	0	0	0	0	0	0	0	0	1	0	0	0	1	0
oestrogen	0	0	1	1	0	0	0	0	0	0	0	0	0	0
patients	1	1	0	1	0	0	0	1	0	0	0	0	0	0
pressure	0	0	0	0	0	0	0	0	0	0	1	0	0	1
rats	0	0	0	0	0	0	0	0	0	0	0	0	1	1
respect	0	0	0	0	0	0	0	1	0	0	0	1	0	0
rise	0	0	0	1	0	0	0	0	0	0	0	0	0	1
study	1	0	1	0	0	0	0	0	1	0	0	0	0	0

- Space complexity of BoW matrix:
 - $O(D * V)$, D is total number of documents and V is vocabulary size
 - Consider D=1million docs, 1k words each, with total V=500k distinct words
 - Hard to store the matrix: half-a-trillion 0/1 elements!
- Zipf's law: each document only has about 10% of vocabulary observed in it
 - Sparse matrix
 - 90% of space is wasted!
- Space efficiency can be greatly improved by only storing the occurred words

First attempt: linked list

- Time complexity analysis
 - $O(|q| * D * |D|)$
 - $|q|$ is the length of query, $|D|$ is the length of a document

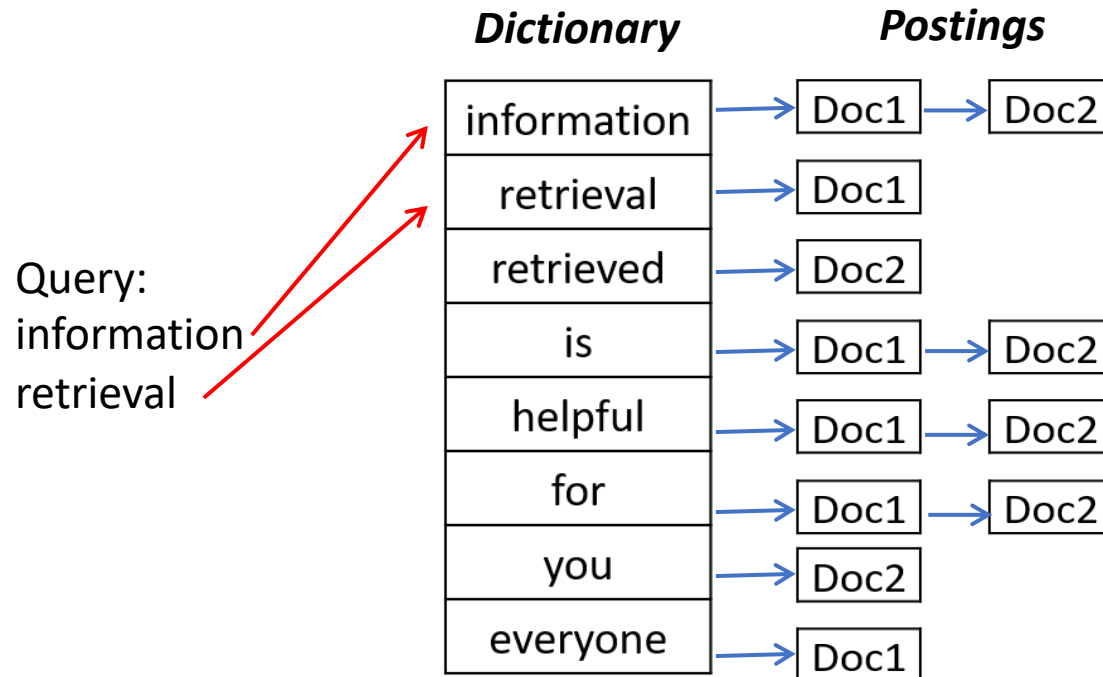
```
doclist = []  
for (wi in q) {  
  for (d in D) {  
    for (wj in d) {  
      if (wi == wj) {  
        doclist += [d];  
        break;  
      }  
    }  
  }  
}  
return doclist;
```

*Bottleneck, since most
of them won't match!*

Two red arrows originate from the text 'Bottleneck, since most of them won't match!'. One arrow points to the 'for (d in D) {' line, and the other points to the 'for (wj in d) {' line, highlighting the nested iteration over documents and their words as the source of inefficiency.

Inverted Index

- Build a look-up table for each word in vocabulary
 - From word to documents



Time complexity:

- $O(|q| * |L|)$, $|L|$ is the average length of posting list
- By Zipf's law, $|L| \ll D$

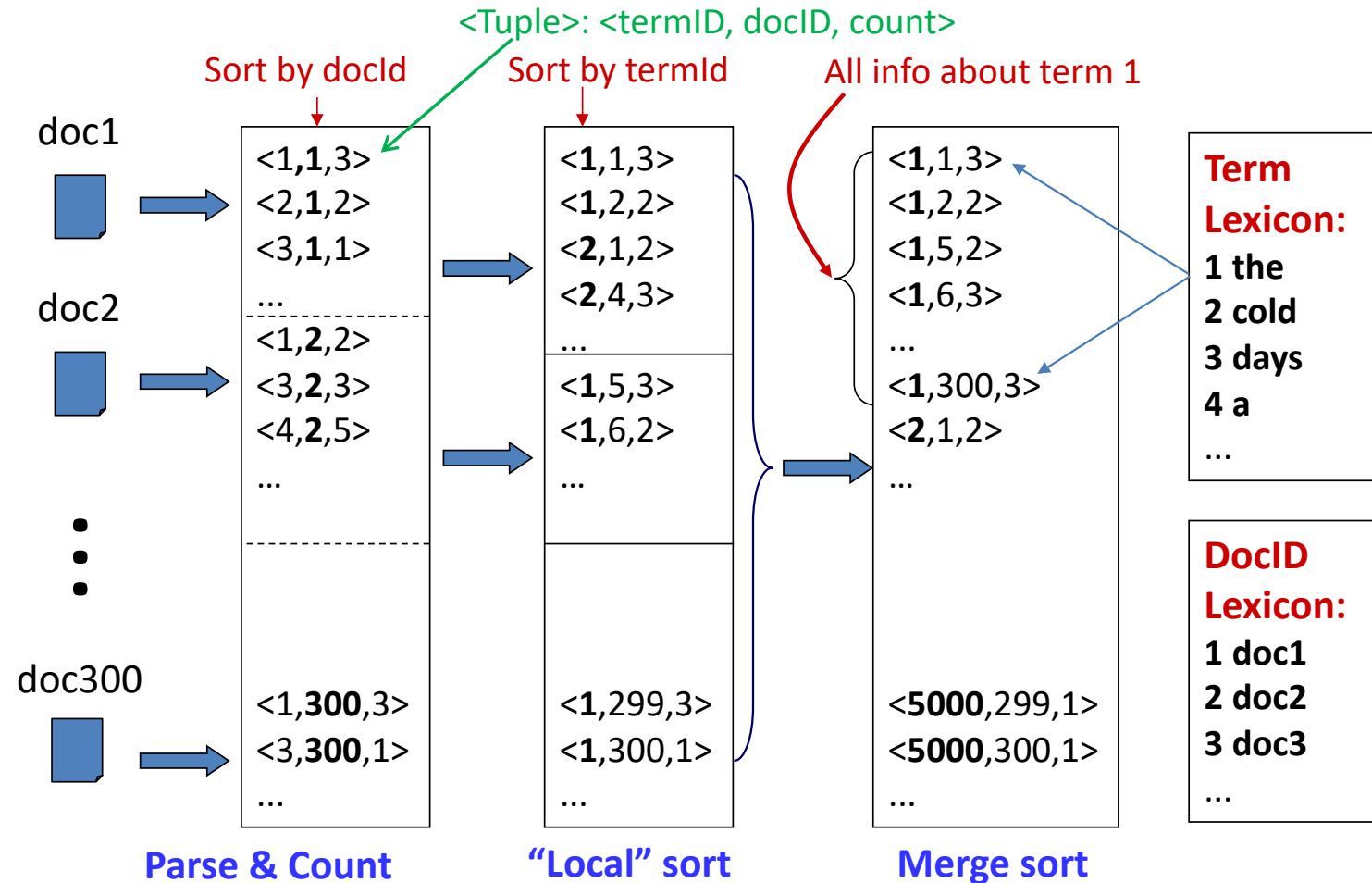
Inverted Index

- Dictionary: modest size
 - Needs fast random access
 - Stay in memory
 - Hash table, B-tree, trie, ...
- Postings: huge
 - Variable size
 - Stay on disk, sequential access is expected
 - In memory, can use linked lists or variable length arrays
 - Contain docID, term freq, term position, ...
 - Compression is needed

Inverted index construction

- Text processing
 - Tokenization, Normalization, Stemming, Stopwords removal
- **Sort:** by terms, then docID
 - Challenges: Document size exceeds memory limit
 - Key steps
 - Local sort: sort by termID
 - For later global merge sort
 - Global merge sort
 - Preserve docID order: for later posting list join
- Dictionary & Postings

Sorting



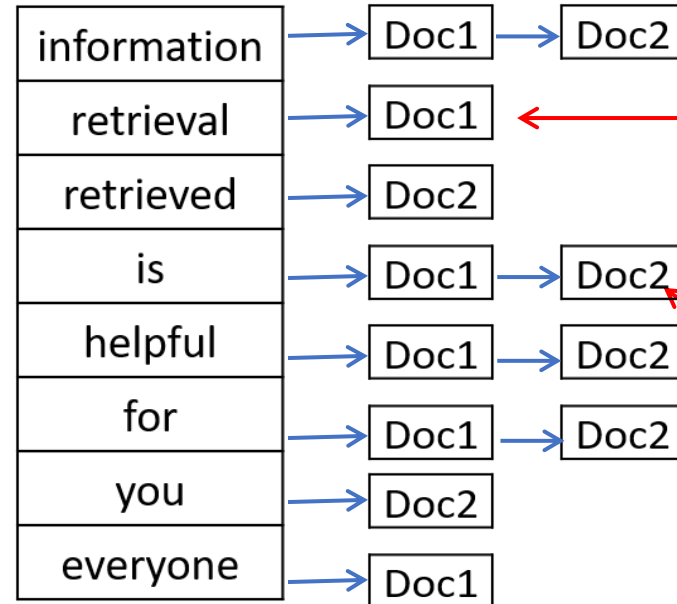
A closer look at inverted index

Approximate search:
e.g., misspelled queries,
wildcard queries

Proximity search:
e.g., phrase queries

Dictionary

Postings



Dynamic index update

Index compression

Dynamic index update

- Periodically rebuild the index
 - Acceptable if change is small over time and penalty of missing new documents is negligible
- Auxiliary index
 - Keep index for new documents in memory
 - Merge to index when size exceeds threshold
 - Increase I/O operation
 - Solution: multiple auxiliary indices on disk, logarithmically merging

Index compression

- Benefits
 - Save storage space
 - Increase cache efficiency
 - Improve disk-memory transfer rate
- Observation of posting files
 - Instead of storing docID in posting, we store gap between docIDs, since they are ordered
 - Zipf's law again:
 - The more frequent a word is, the smaller the gaps are
 - The less frequent a word is, the shorter the posting list is
 - Heavily biased distribution gives us great opportunity of compression!
- ***Information theory***: entropy measures compression difficulty.

Index compression

- Solution

- Fewer bits to encode small (high frequency) integers
- Variable-length coding
 - Unary: $x \geq 1$ is coded as $x-1$ bits of 1 followed by 0, e.g., $3 \Rightarrow 110$; $5 \Rightarrow 11110$
 - γ -code: $x \Rightarrow$ unary code for $1 + \lfloor \log x \rfloor$ followed by uniform code for $x - 2^{\lfloor \log x \rfloor}$ in $\lfloor \log x \rfloor$ bits, e.g., $3 \Rightarrow 101$, $5 \Rightarrow 11001$
 - δ -code: same as γ -code, but replace the unary prefix with γ -code. E.g., $3 \Rightarrow 1001$, $5 \Rightarrow 10101$

► **Table 5.6** Index and dictionary compression for Reuters-RCV1. The compression ratio depends on the proportion of actual text in the collection. Reuters-RCV1 contains a large amount of XML markup. Using the two best compression schemes, γ encoding and blocking with front coding, the ratio compressed index to collection size is therefore especially small for Reuters-RCV1: $(101 + 5.9) / 3600 \approx 0.03$.

data structure	size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
~, with blocking, $k = 4$	7.1
~, with blocking & front coding	5.9
collection (text, xml markup etc)	3600.0
collection (text)	960.0
term incidence matrix	40,000.0
postings, uncompressed (32-bit words)	400.0
postings, uncompressed (20 bits)	250.0
postings, variable byte encoded	116.0
postings, γ encoded	101.0

Compression rate: $(101 + 11.2) / 960 = 11.7\%$

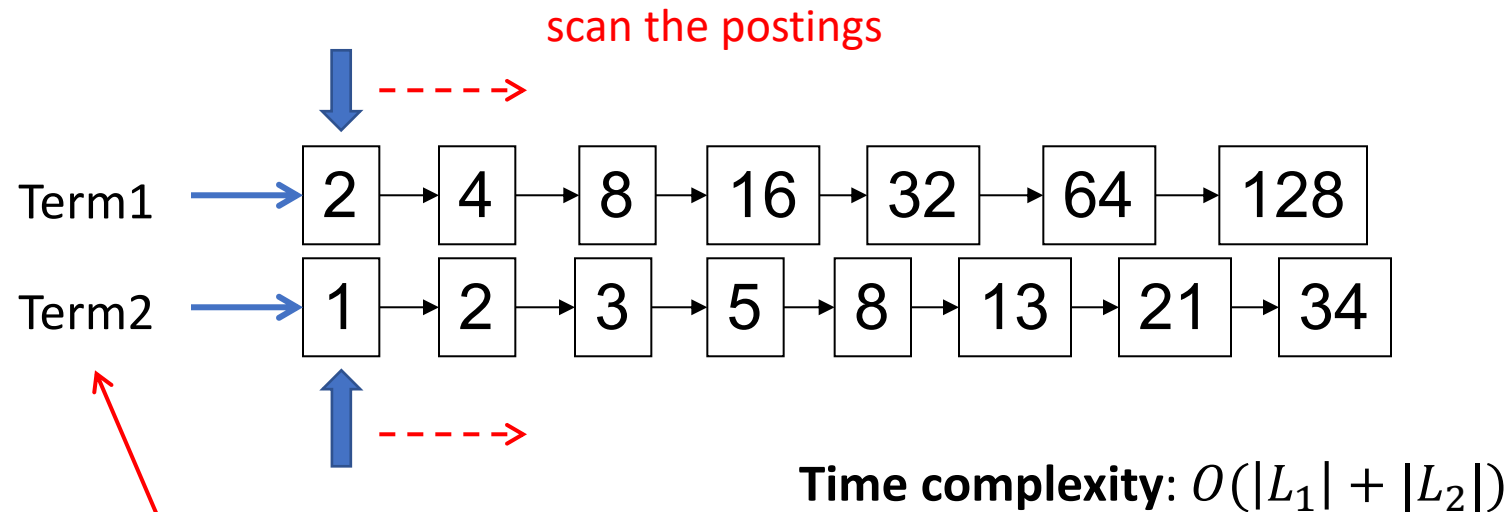
Query processing

- How do we process a query?
- Parse query syntax
 - E.g., Barack AND Obama, orange OR apple
- Perform the same processing procedures as on documents to the input query
 - Tokenization->normalization->stemming->stopwords removal

Query processing

- Procedures
 - Lookup query terms in the dictionary
 - Retrieve the posting lists
 - Operation
 - AND: intersect the posting lists
 - OR: union the posting list
 - NOT: diff the posting list

Query processing: AND operation



Trick for speed-up: when performing multi-way join, starts from lowest frequency term to highest frequency ones

Intersecting two postings lists (a “merge” algorithm)

INTERSECT(p_1, p_2)

```
1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then ADD(answer,  $\text{docID}(p_1)$ )
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return answer
```

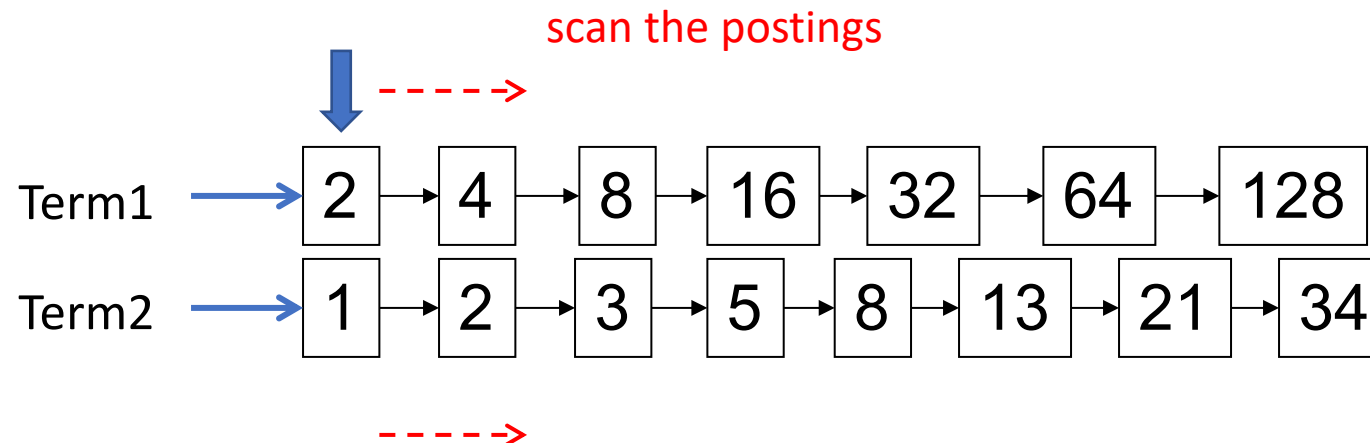
Phrase query

- Query: “information retrieval”
- “*I have information that helps with the retrieval of the treasure*” is not a match.
 - The concept of phrase queries has proven easily understood by users; one of the few “advanced search” ideas that works
 - Many more queries are *implicit phrase queries*
- Solution 1: N-grams (generally does not work for this)
 - Large dictionary size, how to break a long phrase into N-grams?
- We need term positions in documents
 - We can store them in the inverted index

Phrase query

- Generalized postings matching

- Equality condition check with requirement of position pattern between two query terms
 - e.g., $T2.pos - T1.pos = 1$ (T1 must be immediately before T2 in any matched document)
- Proximity query: $|T2.pos - T1.pos| \leq k$



Other information in index

- Document structure
 - Title, abstract, body, bullets, anchor
- Entity annotation
 - Being part of a person's name, location's name

Query spelling correction

- Tolerate the misspelled queries
 - “barck obama” -> “barack obama”
- Principles
 - Of various alternative correct spellings of a misspelled query, choose the ***nearest*** one
 - Of various alternative correct spellings of a misspelled query, choose the ***most common*** one

Query spelling correction: proximity

- Edit distance
 - Minimum number of edit operations required to transform one string to another
 - Insert, delete, replace
 - Tricks for speed-up
 - Fix prefix length (error does not happen on the first letter)
 - Build character-level inverted index, e.g., for length 3 characters
 - Consider the layout of a keyboard
 - E.g., 'u' is more likely to be typed as 'y' instead of 'z'

Query spelling correction: proximity

- Query context
 - “flew form PDX” -> “flew from PDX”
 - Enumerate alternatives for all the query terms
 - Heuristics must be applied to reduce the search space
- Phonetic similarity
 - “herman” -> “Hermann”
 - Phonetic hashing – similar-sounding terms hash to the same value

Summary

- Inverted index for information retrieval
 - Sorting-based index construction
 - Index compression
- Search in inverted index
 - Phrase query
 - Query spelling correction

Readings

- [MRS] Ch 2, Ch 4, Ch 5.